```
import { useState } from "react";

const theme = {
  bg: "#080c10",
  surface: "#0d1117",
  surfaceAlt: "#111820",
  border: "#1e2d3d",
  accent: "#00d4ff",
  accentDim: "#0a3d52",
  accentGlow: "rgba(0,212,255,0.12)",
  warn: "#ff6b35",
  success: "#00e5a0",
  muted: "#4a6070",
  text: "#c9d5e0",
  textBright: "#eaf4ff",
  red: "#ff4466",
};

const code = {
  keyword: "#ff79c6",
  string: "#f1fa8c",
  type: "#8be9fd",
  comment: "#6272a4",
  func: "#50fa7b",
  var: "#bd93f9",
  plain: "#f8f8f2",
  num: "#ffb86c",
};

const SECTIONS = [
  "Overview",
  "Core API",
  "Scanner",
  "Pruner",
  "Ephemeral Core",
  "Vault Adapters",
  "Checkpoint System",
  "Framework Adapters",
  "Audit & Compliance",
  "x402 Payments",
];

const Tag = ({ children, color }) => (
  <span style={{
```

```
    fontSize: 10,
    fontFamily: "'JetBrains Mono', monospace",
    background: color + "22",
    color: color,
    border: `1px solid ${color}44`,
    borderRadius: 3,
    padding: "2px 6px",
    letterSpacing: 1,
    fontWeight: 700,
    textTransform: "uppercase",
  }}>{children}</span>
);

const CodeBlock = ({ lines }) => (
  <div style={{
    background: "#040810",
    border: `1px solid ${theme.border}`,
    borderRadius: 8,
    padding: "16px 20px",
    fontFamily: "'JetBrains Mono', 'Fira Code', monospace",
    fontSize: 12.5,
    lineHeight: 1.75,
    overflowX: "auto",
  }}>
    {lines.map((line, i) => (
      <div key={i} dangerouslySetInnerHTML={{ __html: line }} />
    ))}
  </div>
);

const SectionTitle = ({ children, sub }) => (
  <div style={{ marginBottom: 24 }}>
    <div style={{
      fontSize: 11,
      letterSpacing: 3,
      color: theme.accent,
      fontFamily: "'JetBrains Mono', monospace",
      fontWeight: 700,
      textTransform: "uppercase",
      marginBottom: 8,
    }}>// {sub}</div>
    <h2 style={{
      margin: 0,
      fontSize: 26,
      fontFamily: "'Syne', sans-serif",
      color: theme.textBright,
      fontWeight: 800,
```

```
      letterSpacing: -0.5,
    }}>{children}</h2>
  </div>
);

const ApiCard = ({ name, type, desc, returns, badge }) => (
  <div style={{
    border: `1px solid ${theme.border}`,
    borderRadius: 8,
    marginBottom: 12,
    overflow: "hidden",
    transition: "border-color 0.2s",
  }}
    onMouseEnter={e => e.currentTarget.style.borderColor = theme.accent + "55"}
    onMouseLeave={e => e.currentTarget.style.borderColor = theme.border}
  >
    <div style={{
      padding: "12px 16px",
      background: theme.surfaceAlt,
      borderBottom: `1px solid ${theme.border}`,
      display: "flex",
      alignItems: "center",
      gap: 10,
    }}>
      <span style={{
        fontFamily: "'JetBrains Mono', monospace",
        color: code.func,
        fontSize: 13,
        fontWeight: 700,
      }}>{name}</span>
      {badge && <Tag color={
        badge === "async" ? theme.accent :
        badge === "sync" ? theme.success :
        badge === "event" ? theme.warn : theme.muted
      }>{badge}</Tag>}
      <span style={{ marginLeft: "auto", fontFamily: "monospace", fontSize: 11, c
    </div>
    <div style={{ padding: "12px 16px" }}>
      <p style={{ margin: 0, color: theme.text, fontSize: 13, lineHeight: 1.6 }}>
      {returns && (
        <div style={{ marginTop: 8, fontSize: 12, color: theme.muted }}>
          Returns: <span style={{ color: code.string }}>{returns}</span>
        </div>
      )}
    </div>
  </div>
);
```

```
const AdapterCard = ({ name, icon, status, desc }) => (
  <div style={{
    border: `1px solid ${status === "built-in" ? theme.accent + "44" : theme.bord
    borderRadius: 8,
    padding: "14px 16px",
    background: status === "built-in" ? theme.accentGlow : "transparent",
  }}>
    <div style={{ display: "flex", alignItems: "center", gap: 8, marginBottom: 6
      <span style={{ fontSize: 18 }}>{icon}</span>
      <span style={{ color: theme.textBright, fontWeight: 700, fontSize: 14, font
      <Tag color={status === "built-in" ? theme.success : status === "planned" ?
        {status}
      </Tag>
    </div>
    <p style={{ margin: 0, color: theme.muted, fontSize: 12, lineHeight: 1.5 }}>{
  </div>
);

const OverviewSection = () => (
  <div>
    <SectionTitle sub="What is this">AgentGuard SDK</SectionTitle>
    <p style={{ color: theme.text, lineHeight: 1.8, marginBottom: 32, fontSize: 1
      Open-source security middleware for multi-agent AI systems. Automatically d
      and vaults exposed credentials, API keys, private keys, and seed phrases fr
      at checkpoints or in real-time. Framework-agnostic, enterprise-ready.
    </p>

    <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr 1fr", gap: 12, m
      {[
        { icon: "🔍", title: "Auto-Detection", desc: "Pattern library for 40+ sec
        { icon: "✂️", title: "Checkpoint Pruning", desc: "Batch clean agent state
        { icon: "🔐", title: "Vault Transfer", desc: "Move secrets to env, .env f
        { icon: "🔌", title: "Framework Agnostic", desc: "LangChain, Anthropic, C
        { icon: "📋", title: "Audit Trail", desc: "Compliance-grade logs of every
        { icon: "🧩", title: "Extensible", desc: "Custom pattern registries, vaul
      ].map(c => (
        <div key={c.title} style={{
          border: `1px solid ${theme.border}`,
          borderRadius: 8,
          padding: 16,
          background: theme.surfaceAlt,
        }}>
          <div style={{ fontSize: 22, marginBottom: 8 }}>{c.icon}</div>
          <div style={{ color: theme.textBright, fontWeight: 700, fontSize: 13, m
          <div style={{ color: theme.muted, fontSize: 12, lineHeight: 1.5 }}>{c.d
        </div>
```

```
      ))}
    </div>

    <SectionTitle sub="Package structure">Monorepo Layout</SectionTitle>
    <CodeBlock lines={[
      `<span style="color:${code.comment}">// @agentguard — npm namespace</span>`
      ``,
      `<span style="color:${code.plain}">packages/</span>`,
      `<span style="color:${code.var}">  @agentguard/core</span>      <span style
      `<span style="color:${code.var}">  @agentguard/vault</span>     <span style
      `<span style="color:${code.var}">  @agentguard/patterns</span>  <span style
      `<span style="color:${code.var}">  @agentguard/adapters</span>  <span style
      `<span style="color:${code.var}">  @agentguard/server</span>    <span style
      `<span style="color:${code.var}">  @agentguard/cli</span>       <span style
      `<span style="color:${code.var}">  agentguard</span>            <span style
    ]} />
  </div>
);

const CoreAPISection = () => (
  <div>
    <SectionTitle sub="Entry Point">AgentGuard Client</SectionTitle>
    <CodeBlock lines={[
      `<span style="color:${code.keyword}">import</span> <span style="color:${cod
      ``,
      `<span style="color:${code.keyword}">const</span> <span style="color:${code
      `  <span style="color:${code.plain}">vault:</span> <span style="color:${cod
      `  <span style="color:${code.plain}">patterns:</span> <span style="color:${
      `  <span style="color:${code.plain}">checkpoints:</span> <span style="color
      `  <span style="color:${code.plain}">audit:</span> <span style="color:${cod
      `  <span style="color:${code.plain}">onSecretFound:</span> <span style="col
      `    <span style="color:${code.comment}">// custom hook — alert, notify, et
      `  <span style="color:${code.plain}">}</span>`,
      `<span style="color:${code.plain}">})</span>`,
      ``,
      `<span style="color:${code.comment}">// Wrap any agent call</span>`,
      `<span style="color:${code.keyword}">const</span> <span style="color:${code
      ``,
      `<span style="color:${code.comment}">// Manual checkpoint trigger</span>`,
      `<span style="color:${code.keyword}">await</span> <span style="color:${code
    ]} />

    <div style={{ marginTop: 24 }}>
      {[
        { name: "guard.wrap(fn, state)", badge: "async", type: "Promise<WrappedRe
        { name: "guard.checkpoint(state)", badge: "async", type: "Promise<Checkpo
        { name: "guard.scan(text | object)", badge: "async", type: "Promise<ScanR
```

```
        { name: "guard.prune(state, matches)", badge: "sync", type: "PruneResult"
        { name: "guard.on(event, handler)", badge: "event", type: "void", desc: "
      ].map(a => <ApiCard key={a.name} {...a} />)}
    </div>
  </div>
);

const ScannerSection = () => (
  <div>
    <SectionTitle sub="Detection Engine">Scanner</SectionTitle>
    <p style={{ color: theme.text, fontSize: 13, lineHeight: 1.7, marginBottom: 2
      The scanner accepts raw strings, objects, arrays, or full conversation hist
      It recursively traverses data structures, applies pattern matching, and ret
      with severity, location path, and the redacted form.
    </p>
    <CodeBlock lines={[
      `<span style="color:${code.keyword}">import</span> <span style="color:${cod
      ``,
      `<span style="color:${code.keyword}">const</span> <span style="color:${code
      `  <span style="color:${code.plain}">patterns: builtinPatterns,</span>`,
      `  <span style="color:${code.plain}">deepScan:</span> <span style="color:${
      `  <span style="color:${code.plain}">includeBase64:</span> <span style="col
      `  <span style="color:${code.plain}">contextWindow:</span> <span style="col
      `<span style="color:${code.plain}">})</span>`,
      ``,
      `<span style="color:${code.keyword}">const</span> <span style="color:${code
      `<span style="color:${code.comment}">// result.matches: [{</span>`,
      `<span style="color:${code.comment}">//   type: 'ethereum_private_key',</sp
      `<span style="color:${code.comment}">//   severity: 'critical',</span>`,
      `<span style="color:${code.comment}">//   path: 'messages[2].content',</spa
      `<span style="color:${code.comment}">//   redacted: '0xac0974...••••••••',<
      `<span style="color:${code.comment}">//   confidence: 0.99</span>`,
      `<span style="color:${code.comment}">// }]</span>`,
    ]} />

    <div style={{ marginTop: 24 }}>
      <div style={{ color: theme.textBright, fontWeight: 700, marginBottom: 12, f
      <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 8 }}>
        {[
          ["🔑", "Ethereum / EVM private keys", "critical"],
          ["🌱", "BIP-39 seed phrases (12/24 word)", "critical"],
          ["🔐", "OpenAI / Anthropic / HuggingFace API keys", "high"],
          ["🗝️", "AWS / GCP / Azure credentials", "high"],
          ["🐙", "GitHub tokens (classic + fine-grained)", "high"],
          ["🏦", "Stripe, Twilio, SendGrid keys", "high"],
          ["🗄️", "Database connection strings (Postgres, MySQL, Mongo)", "high"],
          ["🎟️", "JWT tokens", "medium"],
```

```
          ["🔒", "RSA / Ed25519 PEM private keys", "critical"],
          ["🌐", "Generic bearer tokens", "medium"],
        ].map(([icon, name, severity]) => (
          <div key={name} style={{
            border: `1px solid ${theme.border}`,
            borderRadius: 6,
            padding: "10px 12px",
            display: "flex",
            alignItems: "center",
            gap: 8,
          }}>
            <span style={{ fontSize: 14 }}>{icon}</span>
            <span style={{ color: theme.text, fontSize: 12, flex: 1 }}>{name}</sp
            <Tag color={severity === "critical" ? theme.red : severity === "high"
          </div>
        ))}
      </div>
    </div>
  </div>
);

const PrunerSection = () => (
  <div>
    <SectionTitle sub="Memory Sanitization">Pruner</SectionTitle>
    <CodeBlock lines={[
      `<span style="color:${code.keyword}">import</span> <span style="color:${cod
      ``,
      `<span style="color:${code.keyword}">const</span> <span style="color:${code
      `  <span style="color:${code.plain}">strategy:</span> <span style="color:${
      `  <span style="color:${code.plain}">redactWith:</span> <span style="color:
      `  <span style="color:${code.plain}">preserveStructure:</span> <span style=
      `  <span style="color:${code.plain}">vaultOnPrune:</span> <span style="colo
      `<span style="color:${code.plain}">})</span>`,
      ``,
      `<span style="color:${code.keyword}">const</span> <span style="color:${code
      ``,
      `<span style="color:${code.comment}">// PruneStrategy.REDACT   → replaces v
      `<span style="color:${code.comment}">// PruneStrategy.REMOVE   → deletes th
      `<span style="color:${code.comment}">// PruneStrategy.REPLACE  → swaps with
      `<span style="color:${code.comment}">//                          e.g. "{{va
    ]} />
    <div style={{ marginTop: 16, padding: "12px 16px", background: theme.accentGl
      <span style={{ color: theme.accent, fontSize: 12, fontFamily: "monospace" }
      <p style={{ margin: "6px 0 0", color: theme.text, fontSize: 12, lineHeight:
        The most powerful mode. Instead of just redacting, it writes the secret t
        injects a reference token back into the agent state. The agent can later
        when it actually needs the value — minimizing raw key exposure without br
```

```
      </p>
    </div>
  </div>
);

const VaultSection = () => (
  <div>
    <SectionTitle sub="Pluggable Secret Storage">Vault Adapters</SectionTitle>
    <CodeBlock lines={[
      `<span style="color:${code.comment}">// All adapters implement the VaultAda
      `<span style="color:${code.keyword}">interface</span> <span style="color:${
      `  <span style="color:${code.func}">write</span><span style="color:${code.p
      `  <span style="color:${code.func}">read</span><span style="color:${code.pl
      `  <span style="color:${code.func}">list</span><span style="color:${code.pl
      `  <span style="color:${code.func}">delete</span><span style="color:${code.
      `<span style="color:${code.plain}">}</span>`,
    ]} />
    <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 10, margi
      {[
        { name: "EnvVaultAdapter", icon: "🌿", status: "built-in", desc: "Writes
        { name: "DotEnvVaultAdapter", icon: "📄", status: "built-in", desc: "Appe
        { name: "HashiCorpVaultAdapter", icon: "🏛️", status: "built-in", desc: "F
        { name: "AWSSecretsAdapter", icon: "☁️", status: "built-in", desc: "AWS S
        { name: "GCPSecretAdapter", icon: "🔵", status: "planned", desc: "Google
        { name: "AzureKeyVaultAdapter", icon: "🪟", status: "planned", desc: "Azu
        { name: "InMemoryVaultAdapter", icon: "🧠", status: "built-in", desc: "Ep
        { name: "CustomVaultAdapter", icon: "🔧", status: "extensible", desc: "Im
      ].map(a => <AdapterCard key={a.name} {...a} />)}
    </div>
  </div>
);

const CheckpointSection = () => (
  <div>
    <SectionTitle sub="Trigger System">Checkpoints</SectionTitle>
    <p style={{ color: theme.text, fontSize: 13, lineHeight: 1.7, marginBottom: 2
      Checkpoints are the heartbeat of AgentGuard. They define when the scan-prun
      You can use built-in triggers or define custom ones.
    </p>
    <CodeBlock lines={[
      `<span style="color:${code.keyword}">import</span> <span style="color:${cod
      ``,
      `<span style="color:${code.keyword}">const</span> <span style="color:${code
      ``,
      `<span style="color:${code.comment}">// Built-in triggers</span>`,
      `<span style="color:${code.var}">checkpoints</span><span style="color:${cod
      `<span style="color:${code.var}">checkpoints</span><span style="color:${cod
```

```
      `<span style="color:${code.var}">checkpoints</span><span style="color:${cod
      `<span style="color:${code.var}">checkpoints</span><span style="color:${cod
      ``,
      `<span style="color:${code.comment}">// Custom trigger — fire on any condit
      `<span style="color:${code.var}">checkpoints</span><span style="color:${cod
      `  <span style="color:${code.plain}">name:</span> <span style="color:${code
      `  <span style="color:${code.plain}">when:</span> <span style="color:${code
      `  <span style="color:${code.plain}">intercept:</span> <span style="color:$
      `<span style="color:${code.plain}">})</span>`,
    ]} />
    <div style={{ marginTop: 20, display: "grid", gridTemplateColumns: "1fr 1fr 1
      {[
        ["SESSION_END", "After conversation loop ends"],
        ["MEMORY_SAVE", "Before writing to vector store / DB"],
        ["TOOL_CALL_COMPLETE", "After each tool invocation"],
        ["INTERVAL", "Time-based, every N milliseconds"],
        ["MESSAGE_BOUNDARY", "Between each agent message turn"],
        ["CUSTOM", "Any event in your agent system"],
      ].map(([name, desc]) => (
        <div key={name} style={{ border: `1px solid ${theme.border}`, borderRadiu
          <div style={{ fontFamily: "monospace", color: code.var, fontSize: 11, m
          <div style={{ color: theme.muted, fontSize: 11 }}>{desc}</div>
        </div>
      ))}
    </div>
  </div>
);

const AdaptersSection = () => (
  <div>
    <SectionTitle sub="Framework Bridges">Framework Adapters</SectionTitle>
    <CodeBlock lines={[
      `<span style="color:${code.comment}">// ① LangChain — wrap any chain or ag
      `<span style="color:${code.keyword}">import</span> <span style="color:${cod
      `<span style="color:${code.keyword}">const</span> <span style="color:${code
      ``,
      `<span style="color:${code.comment}">// ② Anthropic SDK — intercept messag
      `<span style="color:${code.keyword}">import</span> <span style="color:${cod
      `<span style="color:${code.keyword}">const</span> <span style="color:${code
      `<span style="color:${code.comment}">// messages are scanned before & after
      ``,
      `<span style="color:${code.comment}">// ③ OpenAI SDK</span>`,
      `<span style="color:${code.keyword}">import</span> <span style="color:${cod
      `<span style="color:${code.keyword}">const</span> <span style="color:${code
      ``,
      `<span style="color:${code.comment}">// ④ Express middleware — for agent AI
      `<span style="color:${code.keyword}">import</span> <span style="color:${cod
```

```
      `<span style="color:${code.var}">app</span><span style="color:${code.plain}
      `<span style="color:${code.comment}">// scans req.body and res.body on ever
    ]} />
  </div>
);

const AuditSection = () => (
  <div>
    <SectionTitle sub="Compliance & Observability">Audit Logger</SectionTitle>
    <CodeBlock lines={[
      `<span style="color:${code.comment}">// Every prune event emits a structure
      `<span style="color:${code.plain}">{</span>`,
      `  <span style="color:${code.string}">"eventId"</span><span style="color:${
      `  <span style="color:${code.string}">"timestamp"</span><span style="color:
      `  <span style="color:${code.string}">"agentId"</span><span style="color:${
      `  <span style="color:${code.string}">"trigger"</span><span style="color:${
      `  <span style="color:${code.string}">"secretsFound"</span><span style="col
      `    <span style="color:${code.string}">"type"</span><span style="color:${c
      `    <span style="color:${code.string}">"severity"</span><span style="color
      `    <span style="color:${code.string}">"path"</span><span style="color:${c
      `    <span style="color:${code.string}">"redacted"</span><span style="color
      `    <span style="color:${code.string}">"vaultRef"</span><span style="color
      `  <span style="color:${code.plain}">}],</span>`,
      `  <span style="color:${code.string}">"action"</span><span style="color:${c
      `  <span style="color:${code.string}">"durationMs"</span><span style="color
      `<span style="color:${code.plain}">}</span>`,
    ]} />
    <div style={{ marginTop: 16, display: "grid", gridTemplateColumns: "1fr 1fr",
      {[
        ["Console logger", "Default. Pretty-prints to stdout.", theme.success],
        ["File logger", "JSONL file rotation. Configurable path.", theme.success]
        ["Webhook logger", "POST audit events to any endpoint.", theme.accent],
        ["Custom logger", "Implement LogAdapter interface.", theme.muted],
      ].map(([name, desc, color]) => (
        <div key={name} style={{ border: `1px solid ${theme.border}`, borderRadiu
          <div style={{ color, fontWeight: 700, fontSize: 12, marginBottom: 4 }}>
          <div style={{ color: theme.muted, fontSize: 12 }}>{desc}</div>
        </div>
      ))}
    </div>
  </div>
);

const InfoBox = ({ color, icon, title, children }) => (
  <div style={{
    padding: "14px 16px",
    background: color + "11",
```

```
    border: `1px solid ${color}33`,
    borderRadius: 8,
    marginBottom: 16,
  }}>
    <div style={{ color, fontWeight: 700, fontSize: 12, fontFamily: "monospace",
    <p style={{ margin: 0, color: theme.text, fontSize: 13, lineHeight: 1.65 }}>{
  </div>
);

const EphemeralCoreSection = () => (
  <div>
    <SectionTitle sub="Zero-Knowledge Pruning">Ephemeral Core</SectionTitle>

    <InfoBox color={theme.red} icon="🔴" title="Core Design Constraint">
      The pruner agent must have zero memory of what it pruned. Once a prune cycl
      all credential data is destroyed from the pruner's own execution context. T
      the outcome: <strong style={{color: theme.textBright}}>SUCCESS, FAILED, or
    </InfoBox>

    <CodeBlock lines={[
      `<span style="color:${code.keyword}">import</span> <span style="color:${cod
      ``,
      `<span style="color:${code.comment}">// Runs in an isolated Worker thread —
      `<span style="color:${code.keyword}">const</span> <span style="color:${code
      `  <span style="color:${code.plain}">isolate:</span> <span style="color:${c
      `  <span style="color:${code.plain}">zeroOnComplete:</span> <span style="co
      `  <span style="color:${code.plain}">noLog:</span> <span style="color:${cod
      `  <span style="color:${code.plain}">auditSchema:</span> <span style="color
      `<span style="color:${code.plain}">})</span>`,
    ]} />

    <div style={{ marginTop: 24 }}>
      <div style={{ color: theme.textBright, fontWeight: 700, fontSize: 13, margi
      {[
        { step: "01", title: "Worker Spawn", desc: "Each prune cycle spawns a fre
        { step: "02", title: "Scan & Vault Atomically", desc: "Inside the Worker:
        { step: "03", title: "Buffer Zeroing", desc: "After vault.write() confirm
        { step: "04", title: "Worker Terminate", desc: "The Worker posts back onl
        { step: "05", title: "Audit Log (Outcome Only)", desc: "The parent logs a
      ].map(s => (
        <div key={s.step} style={{ display: "flex", gap: 16, marginBottom: 14 }}>
          <div style={{
            width: 32, height: 32, borderRadius: 6, flexShrink: 0,
            background: theme.accentGlow, border: `1px solid ${theme.accent}44`,
            display: "flex", alignItems: "center", justifyContent: "center",
            fontFamily: "monospace", fontSize: 11, color: theme.accent, fontWeigh
          }}>{s.step}</div>
```

```
            <div>
              <div style={{ color: theme.textBright, fontSize: 13, fontWeight: 700,
              <div style={{ color: theme.muted, fontSize: 12, lineHeight: 1.6 }}>{s
            </div>
          </div>
        ))}
      </div>

      <div style={{ marginTop: 24 }}>
        <div style={{ color: theme.textBright, fontWeight: 700, fontSize: 13, margi
        <CodeBlock lines={[
          `<span style="color:${code.comment}">// AuditSchema.OUTCOME_ONLY — what g
          `<span style="color:${code.plain}">{</span>`,
          `  <span style="color:${code.string}">"eventId"</span><span style="color:
          `  <span style="color:${code.string}">"timestamp"</span><span style="colo
          `  <span style="color:${code.string}">"outcome"</span><span style="color:
          `  <span style="color:${code.string}">"secretCount"</span><span style="co
          `  <span style="color:${code.string}">"durationMs"</span><span style="col
          `<span style="color:${code.plain}">}</span>`,
          ``,
          `<span style="color:${code.comment}">// ✗ NEVER logged:</span>`,
          `<span style="color:${code.comment}">// secret value, redacted form, key
          `<span style="color:${code.comment}">// pattern name, agent session ID, v
        ]} />
      </div>

      <InfoBox color={theme.success} icon="✅" title="Why secretCount is safe to lc
        A count like "2 secrets found" reveals nothing about what those secrets are
        or which agent produced them. It exists purely for operational alerting — e
        consistently leaks 10+ secrets per session, you want to know that without k
        If even this is too much for your threat model, set <code style={{color: th
      </InfoBox>
    </div>
  );

  const X402Section = () => (
    <div>
      <SectionTitle sub="Agent Micropayments">x402 Payment Layer</SectionTitle>

      <InfoBox color={theme.warn} icon="⚡" title="What is x402?">
        x402 is an HTTP-native micropayment standard (pioneered by Coinbase) built
        "Payment Required" status code. When an agent hits a guarded endpoint with
        it receives a 402 with machine-readable payment instructions. The agent pay
        retries with proof, and the call proceeds. No subscriptions, no API keys fo
      </InfoBox>

      <div style={{ marginTop: 4 }}>
```

```
  <div style={{ color: theme.textBright, fontWeight: 700, fontSize: 13, margi
  {[
    { step: "①", color: theme.muted, label: "Agent calls AgentGuard prune en
    { step: "②", color: theme.warn, label: "Server returns HTTP 402", detail
    { step: "③", color: theme.accent, label: "Agent constructs x402 payment",
    { step: "④", color: theme.accent, label: "Agent retries with proof", deta
    { step: "⑤", color: theme.success, label: "Server verifies on-chain & pr
  ].map(s => (
    <div key={s.step} style={{
      display: "flex", gap: 12, marginBottom: 10,
      padding: "10px 14px",
      border: `1px solid ${theme.border}`,
      borderRadius: 7,
      background: theme.surfaceAlt,
    }}>
      <span style={{ color: s.color, fontFamily: "monospace", fontWeight: 700
      <div>
        <div style={{ color: theme.textBright, fontSize: 12, fontWeight: 600,
        <div style={{ color: theme.muted, fontSize: 11, fontFamily: "monospac
      </div>
    </div>
  ))}
</div>

<div style={{ marginTop: 24 }}>
  <div style={{ color: theme.textBright, fontWeight: 700, fontSize: 13, margi
  <CodeBlock lines={[
    `<span style="color:${code.keyword}">import</span> <span style="color:${c
    `<span style="color:${code.keyword}">import</span> <span style="color:${c
    ``,
    `<span style="color:${code.keyword}">const</span> <span style="color:${co
    `  <span style="color:${code.plain}">guard,</span>`,
    `  <span style="color:${code.plain}">payment: {</span>`,
    `    <span style="color:${code.plain}">enabled:</span> <span style="color
    `    <span style="color:${code.plain}">network:</span>  <span style="colo
    `    <span style="color:${code.plain}">token:</span>    <span style="colo
    `    <span style="color:${code.plain}">payTo:</span>    <span style="colo
    `    <span style="color:${code.plain}">pricing: {</span>`,
    `      <span style="color:${code.string}">'prune'</span><span style="colo
    `      <span style="color:${code.string}">'scan'</span><span style="color
    `      <span style="color:${code.string}">'checkpoint'</span><span style=
    `    <span style="color:${code.plain}">},</span>`,
    `    <span style="color:${code.plain}">freeQuota:</span> <span style="col
    `  <span style="color:${code.plain}">}</span>`,
    `<span style="color:${code.plain}">})</span>`,
  ]} />
</div>
```

```
    <div style={{ marginTop: 24 }}>
      <div style={{ color: theme.textBright, fontWeight: 700, fontSize: 13, margi
      <CodeBlock lines={[
        `<span style="color:${code.keyword}">import</span> <span style="color:${c
        ``,
        `<span style="color:${code.comment}">// The agent client handles 402 nego
        `<span style="color:${code.keyword}">const</span> <span style="color:${co
        `  <span style="color:${code.plain}">endpoint:</span>   <span style="colo
        `  <span style="color:${code.plain}">agentWallet:</span> <span style="col
        `  <span style="color:${code.plain}">maxPriceUSDC:</span><span style="col
        `<span style="color:${code.plain}">})</span>`,
        ``,
        `<span style="color:${code.comment}">// Transparent — just call it. 402 →
        `<span style="color:${code.keyword}">const</span> <span style="color:${co
      ]} />
    </div>

    <div style={{ marginTop: 20, padding: "14px 16px", background: theme.accentGl
      <div style={{ color: theme.accent, fontWeight: 700, fontSize: 12, fontFamil
      <p style={{ margin: 0, color: theme.text, fontSize: 12, lineHeight: 1.65 }}
        Your Buzz agents already carry ERC-8004 onchain identity. That same ident
        No human operator needs to fund each agent individually — agents earn via
        and spend via x402 for services like AgentGuard. Fully autonomous agent e
      </p>
    </div>

    <div style={{ marginTop: 20 }}>
      <div style={{ color: theme.textBright, fontWeight: 700, fontSize: 13, margi
      <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 8 }}>
        {[
          ["Per-call", "Default. $0.0001/prune. Pure pay-as-you-go. Best for spor
          ["Free quota", "First N calls free per agent wallet. Great for adoption
          ["Volume discount", "Negotiated rate for high-throughput agent fleets.
          ["Self-hosted free", "Deploy your own AgentGuard instance. No payments.
        ].map(([name, desc, color]) => (
          <div key={name} style={{ border: `1px solid ${theme.border}`, borderRad
            <div style={{ color, fontWeight: 700, fontSize: 12, marginBottom: 4 }
            <div style={{ color: theme.muted, fontSize: 12, lineHeight: 1.5 }}>{d
          </div>
        ))}
      </div>
    </div>
  </div>
);

const SECTION_CONTENT = {
```

```
  "Overview": <OverviewSection />,
  "Core API": <CoreAPISection />,
  "Scanner": <ScannerSection />,
  "Pruner": <PrunerSection />,
  "Ephemeral Core": <EphemeralCoreSection />,
  "Vault Adapters": <VaultSection />,
  "Checkpoint System": <CheckpointSection />,
  "Framework Adapters": <AdaptersSection />,
  "Audit & Compliance": <AuditSection />,
  "x402 Payments": <X402Section />,
};

export default function App() {
  const [active, setActive] = useState("Overview");

  return (
    <div style={{
      background: theme.bg,
      minHeight: "100vh",
      fontFamily: "'Inter', system-ui, sans-serif",
      display: "flex",
      flexDirection: "column",
    }}>
      <link href="https://fonts.googleapis.com/css2?family=Syne:wght@700;800&fami

      {/* Header */}
      <div style={{
        borderBottom: `1px solid ${theme.border}`,
        padding: "16px 32px",
        display: "flex",
        alignItems: "center",
        gap: 16,
        position: "sticky",
        top: 0,
        background: theme.bg + "ee",
        backdropFilter: "blur(12px)",
        zIndex: 10,
      }}>
        <div style={{
          width: 32, height: 32, borderRadius: 8,
          background: `linear-gradient(135deg, ${theme.accent}, #0070ff)`,
          display: "flex", alignItems: "center", justifyContent: "center",
          fontSize: 16,
        }}>🛡️</div>
        <div>
          <div style={{
            fontFamily: "'Syne', sans-serif",
```

```
      fontWeight: 800,
      fontSize: 18,
      color: theme.textBright,
      letterSpacing: -0.5,
    }}>AgentGuard</div>
    <div style={{ fontSize: 11, color: theme.muted, fontFamily: "monospace"
  </div>
  <div style={{ marginLeft: "auto", display: "flex", gap: 8 }}>
    <Tag color={theme.success}>open source</Tag>
    <Tag color={theme.accent}>TypeScript</Tag>
    <Tag color={theme.warn}>Python soon</Tag>
  </div>
</div>

<div style={{ display: "flex", flex: 1 }}>
  {/* Sidebar */}
  <div style={{
    width: 200,
    borderRight: `1px solid ${theme.border}`,
    padding: "24px 0",
    flexShrink: 0,
    position: "sticky",
    top: 65,
    height: "calc(100vh - 65px)",
    overflowY: "auto",
  }}>
    {SECTIONS.map(s => (
      <button
        key={s}
        onClick={() => setActive(s)}
        style={{
          display: "block",
          width: "100%",
          textAlign: "left",
          padding: "9px 24px",
          background: active === s ? theme.accentGlow : "transparent",
          border: "none",
          borderLeft: `2px solid ${active === s ? theme.accent : "transpare
          color: active === s ? theme.accent : theme.muted,
          fontSize: 12.5,
          fontWeight: active === s ? 700 : 400,
          cursor: "pointer",
          transition: "all 0.15s",
          fontFamily: "inherit",
        }}
      >{s}</button>
    ))}
```

```
        </div>

        {/* Main content */}
        <div style={{ flex: 1, padding: "32px 40px", maxWidth: 800, overflowY: "a
          {SECTION_CONTENT[active]}
        </div>
      </div>
    </div>
  );
}
```